Quotation

What it is

A quotation is the agreement: what a vendor said it would charge. Upload the document, the backend extracts its priced service lines, a reviewer corrects them, and committing turns them into the vendor's **rate card** — the thing every invoice line is later reconciled against.

The reviewer's job is the point. Extraction is a model reading a PDF, so the review page is where a human fixes what it got wrong, before those numbers become the reference every future invoice is judged by.

Routes

Path	Purpose
`/quotation/add`	Upload one or more quotation files (`vendorId` optional)
`/quotation/review`	Review and commit the vendor's uncommitted extractions (`vendorId` required)

Committed quotations are visible as a vendor's table at `/vendor/$vendorId`. There is no cross-vendor view, and no way to open a single committed quotation.

Upload

Driven by `use-quotation-submit.ts`. Only files that finished uploading take part.

Before anything else, it checks `GET /source/uncommitted?vendorId=` — if the vendor already has an in-progress quotation, a dialog offers **Discard & Continue**, because starting a new upload destroys it.

Then the saga:

```
confirm-many → extract-metadata → [mixed-type gate] → validate → [rejected gate] → [warnings gate] → /quotation/review
```

Step	Endpoint
Link the files to the vendor	`POST /files/confirm-many`
Create documents from files	`POST /files/extract-metadata`
Financial gate + vendor match	`POST /source/documents/:id/validate`

Three gates can stop it:

- **Mixed document types** — a quotation batch must be one type; the gate compares the extracted `attributeSet` across the batch.
- **Rejected files** — `validate` always answers 200; rejection is a `status: "rejected"` field with a server-authored reason, not a 4xx. When *every* rejected file is non-financial the dialog says so outright; otherwise it stays neutral and lets each file's reason speak.

- **Warnings** — continue, or discard. Discard here abandons the **whole batch** (`POST /source/discard` takes only a `vendorId`), because quotations have no per-document discard. The dialog says so: "along with the rest of this upload".

Navigation needs only `vendorId` — the review page finds the documents itself. That's the structural difference from invoices, which must wait for extraction to return document ids before they can navigate.

Review

`/quotation/review` reads `GET /source/uncommitted?vendorId=` and walks the files one at a time.

Documents arrive `validated`, and `use-document-advancer.ts` stages each one automatically (`POST /source/documents/:id/extract-services`, re-runnable, all docs in parallel; one failure doesn't block the rest). Until a document reaches `staged` the page shows a dedicated state rather than the editor — extracting, extraction failed, or "no rules to review" (with a distinct message for a non-financial file, since that's the common case).

Editing

Two layouts over the same content — **Split** (list + editor) and **Cards** — remembered in `localStorage` under `quotation-review-view`. The header's search **is** wired.

Per service line: **Name** is read-only; **Label**, **Pricing context**, and one field per attribute are editable, typed and required according to the attribute set. Invalid fields light up immediately on mount, so a bad extraction is visible before you touch it. "View source" opens the originating chunk (`GET /source/content/:contentId`) with the matched text highlighted.

You can **edit and delete** service lines. You cannot add one — there's no create action and no endpoint.

Saving — two phases

Saving happens in three stages:

1. **Typing saves nothing.** Edits live in a draft map, deletions in a local set. A draft that matches the server value is dropped rather than stored, so drafts only ever hold real diffs.
2. **Next File / Finish Review** → `POST /source/documents/:id/mark-ready` with `{ services, deletedServiceIds }`. This persists the edits *and* parks the file as `ready_to_commit`. Per its docblock: "*No graph write — the actual commit is deferred to `commit-all`.*"
3. **Save & Complete** on the completion screen → `POST /source/commit-all`. **This is the only graph write.**

So a file can be fully reviewed and still not be a rate card. Going back (`unmark-ready`) drops that file's pending edits by design. `commit-all` is partial-failure tolerant — it reports `committed` and `failed` separately, and the UI toasts the failures and stays put rather than navigating.

Committing invalidates the vendor's quotations key, which is what makes the new lines appear on `/vendor/$vendorId`.

Endpoints

Purpose	Method + path
Uncommitted for a vendor	`GET /source/uncommitted?vendorId=`
Latest uncommitted (org-wide)	`GET /source/uncommitted/latest`
Source chunk	`GET /source/content/:contentId`
Discard (vendor-wide)	`POST /source/discard`
Validate	`POST /source/documents/:id/validate`
Extract services	`POST /source/documents/:id/extract-services`
Save + park a file	`POST /source/documents/:id/mark-ready`
Reopen a parked file	`POST /source/documents/:id/unmark-ready`
Commit the batch	`POST /source/commit-all`

`GET /source/uncommitted/latest` is what drives the "previous quotation incomplete" banner on the dashboard and the add page.

Data

There is no `price` field. Pricing lives in a per-document `attributes` bag, described by an `AttributeSet`:

```
export type AttributeDef = {
  key: string
  label: string
  type: "string" | "number"   // picks the input
  nullable: boolean           // whether it may be cleared
}
```

That set is the schema for everything: it generates the edit form's fields, their validation, and the quotations table's columns. A vendor with no committed services has no attribute set at all.

A **tier** is a variant line of one service ("20ft" / "40ft"), carrying its own attribute bag.

Two status axes

```
// the document's own pipeline status
type DocumentStatus =
  | "pending" | "extracted" | "validated" | "staged"
  | "rejected" | "committed" | "discarded"

// the staged row's sub-status
type StagedStatus = "staged" | "ready_to_commit" | "committed" | "superseded"
```

These are two separate fields. `DocumentStatus` is the document's position in the pipeline; `StagedStatus` (carried by a field named `reviewStatus`) is the state of its staged row once extraction produces one. The value `staged` exists in both: a document at the `staged` step has a staged row that also starts at `staged`, and advances to `ready_to_commit` when the reviewer parks it.

Note that this `reviewStatus` is a different field from the one invoices use (theirs is `"Pending" | "Completed"`); the two share a name only.

Both features narrow one shared `DOCUMENT_STATUSES` union to their own lifecycle: quotations get `staged` / `rejected` / `discarded`, invoices get `processing` / `matching` / `matched`.

Quotations have no status badge UI. Status surfaces only as review-page states and the file bar's dot strip.

The vendor quotations table

Lives on `/vendor/$vendorId`. Columns are **generated from the attribute set**, so the table adapts per vendor: index, Quotation (title + code), Tier (only when some group is tiered), one column per attribute (`remarks` skipped — Conditions already carries it), Conditions.

Pagination is server-side (`limit` / `offset`, URL-persisted).

Numbers render plainly, without unit semantics: an insurance premium of `0.10` (meaning 0.10%) shows as `0.1`. Unit context survives only as prose in Conditions.

File index

`features/quotations/`

File	Purpose
`components/add-quotation-form.tsx`	Upload page: dropzone, vendor select, submit
`components/confirm-discard-upload-dialog.tsx`	"You have an incomplete quotation" gate
`components/incomplete-quotation-banner.tsx`	Dashboard/add-page banner off `uncommitted/latest`
`components/review/rules-review.tsx`	Review page shell + the per-status body
`components/review/rules-review-context.tsx`	Drafts, deletions, validation, mark-ready, commit
`components/review/review-file-bar.tsx`	File n of N, progress dots, Re-extract
`components/review/review-footer.tsx`	Discard / Previous File / Next File · Finish Review
`components/review/services-list.tsx`	Split-view list of services
`components/review/service-edit-form.tsx`	Split-view editor pane
`components/review/cards-review.tsx`	Cards layout
`components/review/service-detail.tsx`	One service: fields, View source, Delete
`components/review/service-form.tsx`	The dynamic field body, driven by the attribute set
`components/review/source-chunk-panel.tsx`	Source chunk with the extracted text highlighted
`components/review/confirm-delete-service-dialog.tsx`	Pure confirm for removing a line
`components/review/discard-review-dialog.tsx`	Pure confirm for discarding the batch
`components/review/review-complete.tsx`	All-files-reviewed screen; Save & Complete
`components/review/no-services-state.tsx`	Empty state for a document with nothing to review
`components/table/quotations-table.tsx`	The vendor's committed quotations; builds dynamic columns
`components/table/quotations-toolbar.tsx`	Count, filter, search, Add Rule, Add Quotation
`components/table/attribute-cell.tsx`	Renders one attribute per tier, row-aligned
`hooks/use-quotation-submit.ts`	The upload saga and its three gates
`hooks/use-document-advancer.ts`	Auto-stages validated docs on the review page
`services/quotations.ts`	The vendor's committed services (the table's read)

File	Purpose
`services/rules-review.ts`	Uncommitted, validate, extract, mark-ready, commit, discard
`mappers/quotations-mappers.ts`	API service groups → table rows
`mappers/rules-review-mappers.ts`	`ReviewService` ⇄ form values; builds the dynamic schema
`lib/document-metadata.ts`	Reads attributeSet / documentKind / isFinancial off metadata
`lib/invalid-files-copy.ts`	Per-reason copy for the invalid-files dialog
`lib/service-search.ts`	Review-page search predicate
`lib/highlight-source.ts`	Splits a chunk around the matched source text
`store/discard-review-dialog.ts`	Discard-confirm open state (vendor-scoped)
`types.ts`	Wire types: services, tiers, attribute sets, both status unions